Chrono Rift
OS Semester Project Report

Roll Number	Name
24i-0617	Abdullah Nadeem
24i-0838	Zainab Nisar

CS 2006 Operating Systems - Spring 2026

Abstract

Chrono Rift is a UNIX-based multiplayer tactical game developed as a semester project to demonstrate practical implementation of core Operating Systems concepts. The project is architecturally decomposed into three strictly isolated UNIX processes the Game Arbiter, the Human Input Process, and the NPC Automation Process, which communicate exclusively through POSIX shared memory. Synchronisation is enforced using unnamed semaphores and process-shared mutexes embedded directly inside the shared memory region, eliminating any reliance on pipes or FIFOs as required by the project specification.

The key OS concepts implemented and demonstrated include:

- Process creation and isolation using fork() and execl()
- Multi-threading via POSIX pthreads with one thread per entity
- Inter-process communication (IPC) via POSIX shared memory (shm_open / mmap)
- Synchronisation using unnamed semaphores and process-shared robust mutexes
- Custom stamina-based CPU scheduling algorithm
- Deadlock detection and recovery via a background watchdog thread
- Custom first-fit contiguous memory allocator for inventory management
- Asynchronous signal handling using SIGUSR1, SIGUSR2, SIGALRM, SIGTERM, SIGCONT
- Real-time graphical rendering via SFML on a dedicated render thread
- Docker-based Ubuntu build environment with X11 forwarding

The system successfully sustains stable synchronised execution under configurations of up to 4 players against 9 simultaneous enemies, with individual action turnaround times consistently below 5 milliseconds.

1. Introduction

Chrono Rift is an advanced multi-process tactical game designed to demonstrate core Operating Systems concepts. The project involves an asymmetric architecture where a human player engages against computer-controlled entities (NPCs). The entire game operates using strict multi-threading, inter-process communication (IPC) via shared memory, and sophisticated synchronization primitives to avoid race conditions and deadlocks.

1.1 Objectives

- Demonstrate process isolation through separate UNIX processes with no shared address space except via explicit shared memory
- Implement IPC strictly through POSIX shared memory, as prohibited by the no-pipes/FIFO constraint
- Prevent race conditions using semaphores and mutexes embedded in shared memory
- Simulate a custom CPU scheduling algorithm using a stamina-based priority model
- Detect and recover from deadlock at runtime without terminating the process
- Implement contiguous memory allocation with spill-to-storage on overflow
- Use UNIX signals for asynchronous cross-process communication
- Integrate a real-time graphical interface that reads shared state safely

1.2 Technologies Used

Technology	Purpose
C++ (g++)	Primary implementation language
POSIX Threads (pthreads)	Multi-threading for player and NPC entities
POSIX Shared Memory (shm_open/mmap)	Inter-process communication
Unnamed Semaphores	Shared memory access synchronisation
Robust Process-Shared Mutexes	Artifact lock management with owner-death recovery
UNIX Signals	Asynchronous events -stun, ultimate, shutdown
SFML 2.x	Real-time graphical rendering
Docker + Ubuntu 22.04	Reproducible build and execution environment
X11 Forwarding	GUI rendering from inside container to host display

2. Turnaround Time Analysis

1. Scheduling Model Overview

Chrono Rift implements a Temporal Stamina-Based Scheduler -a custom CPU scheduling algorithm inspired by real-time OS scheduling. Unlike classical algorithms (Round Robin, SJF, FCFS), this model grants CPU time not by time-slice expiry, but by a readiness threshold accumulated over time.

Each entity (player thread or NPC thread) runs in a continuous loop, incrementing its 'stamina' counter by its 'speed' value every cycle (every 1 second of sleep). Once stamina crosses its ceiling, the entity enters the **Ready State**, the OS scheduling equivalent of a process moving from the *Waiting Queue* to the *Ready Queue*.

Process Arrival	Stamina reaching 100 (Player) / 150 (NPC)
Ready Queue	All threads spin-waiting on 'turn_mutex'
CPU Dispatch	‘pthread_mutex_timedlock()' succeeds
Critical Section	Action execution (attack, heal, etc.)
Context Switch	'pthread_mutex_unlock()' releasing the CPU
Blocking (I/O Burst)	Human thread waiting for GUI key input

2. TAT Definitions

TAT = $T_{completion} - T_{arrival}$
WT = $T_{dispatch} - T_{arrival}$
BT = $T_{completion} - T_{dispatch}$

Where:
-$\mathbf{T_{arrival}}$: Recorded in 'human_process.cpp' / 'npc_process.cpp' via 'get_time_now()' the moment stamina hits threshold
- $\mathbf{T_{dispatch}}$: The moment 'pthread_mutex_timedlock' returns 0 (turn_mutex acquired)
- $\mathbf{T_{completion}}$: Recorded immediately after stamina is reset to 0 and 'turn_mutex' is released

3. Two Distinct Process Behaviours: CPU-Bound vs. I/O-Bound

NPC Threads (CPU-Bound):
The 'npc_process' threads choose actions via 'rand()' and immediately execute. There is no blocking I/O. TAT reflects only scheduling contention; how long a thread waits to acquire 'turn_mutex' while other threads hold it. This is a **pure CPU-bound workload**

Player Threads (I/O-Bound):
The 'human_process' threads block indefinitely in a polling loop ('usleep(50000)') waiting for the SFML 'render_thread' to write a 'gui_input' value via shared memory. This is **I/O-bound behaviour**, where the "I/O device" is the human keyboard. The thread is in the Waiting State until input arrives, then transitions back to the Ready State to compete for 'turn_mutex'
Since action execution (Burst Time) is computationally trivial (< 0.1 ms of pure memory writes to shared memory), the dominant factor in TAT is always the Wait Time

4.Observed TAT - Real Data from 'performance_log.csv'

Per-Entity Statistics (Computed from 63 Total Turns Logged)

Entity	**Turns**	**Min TAT**	**Max TAT**	**Avg TAT**
Crono	19	50.10 ms	8,706.09 ms	3,082 ms
Marle	13	154.36 ms	19,122.31 ms	6,594 ms
Frog	11	613.04 ms	31,378.33 ms	8,067 ms
Lucca	10	622.93 ms	30,554.08 ms	9,028 ms
All Players	53	50.10 ms	31,378.33 ms	6,100 ms
Lavos Spawn 3	3	0.01 ms	0.03 ms	0.02 ms
Lavos Spawn 4	7	0.01 ms	0.05 ms	0.02 ms
Lavos Spawn 6	1	0.11 ms	0.11 ms	0.11 ms
Lavos Spawn 7	5	0.01 ms	5,006.80 ms	1,001 ms
Lavos Spawn 8	3	0.01 ms	20,012.44 ms	6,671 ms

Lavos Spawn 5	1	10,012.26 ms	10,012.26 ms	10,012 ms
All NPCs (baseline)	30 turns (no interrupt)	0.01 ms	0.11 ms	0.022 ms
All NPCs (interrupted)	9 turns (signal events)	5,001 ms	20,012 ms	7,569 ms

5. Analysis by Category

Category A: NPC Baseline -Scheduling Efficiency

Lavos Spawn 4, 0.02 ms
Lavos Spawn 7, 0.01 ms
Lavos Spawn 3, 0.03 ms
Average across 30 uninterrupted NPC turns: 0.022 ms

These values represent the **true overhead** of our OS design - the combined cost of:
1. Acquiring a futex-backed 'pthread_mutex' (POSIX Mutex → Kernel Futex)
2. Writing action results to 'mmap' shared memory (zero-copy, in-place)
3. Releasing the mutex and yielding the CPU

0.022 ms is exceptionally efficient, confirming that shared memory IPC introduces negligible latency compared to message-passing alternatives like pipes or sockets (which would incur kernel copy overhead on every message).

Category B: Player TAT = I/O-Bound Blocking

Crono: 50.10 ms → 8,706 ms (Avg: 3,082 ms)
Marle: 154.36 ms → 19,122 ms (Avg: 6,594 ms)
Frog: 613.04 ms → 31,378 ms (Avg: 8,067 ms)
Lucca: 622.93 ms → 30,554 ms (Avg: 9,028 ms)

These TATs represent **Human Reaction Time** captured as an OS scheduling metric. The thread enters the Ready State (stamina ≥ 100), records$T_{arrival}$, and immediately blocks in a keyboard polling loop - transitioning from Ready → Waiting (I/O Block). The thread only re-enters the Ready State once the player presses a key.

- **Fastest observed** - *Crono, 50.10 ms*: The player had their finger ready and responded almost immediately.

- **Slowest observed** - *Frog, 31,378 ms* (31.4 seconds): The player was reading the UI/menu before deciding. This is a valid I/O burst in OS terms - the CPU thread is correctly parked in a Waiting State, consuming 0% CPU during this time.
- **Crono's consistent low TAT** (avg 3,082 ms vs Marle/Frog/Lucca at 6,000–9,000 ms) indicates a single player controlling Crono actively while others are on longer deliberation cycles.

Category C: Signal-Induced Interrupts - Quantifying OS Signal Latency
11:13:34, Lavos Spawn 5, 10012.26 ms -SIGSTOP / SIGCONT Ultimate (10s freeze)
11:13:52, Lavos Spawn 7, 5006.80 ms   -SIGUSR1/SIGUSR2 Stun (3s) + Mutex wait
11:13:59, Lavos Spawn 8, 20012.44 ms -Two consecutive Ultimates (2×10s)
11:14:02, Lavos Spawn 9, 5001.40 ms   - Stun interrupt
11:14:11, Lavos Spawn 10, 8014.15 ms - Partial Ultimate overlap

These entries directly demonstrate UNIX Signal behaviour as an OS scheduling interrupt mechanism:

'SIGSTOP' (Ultimate Ability):
The Arbiter calls 'kill(npc_process_pid, SIGSTOP)', which causes the Linux kernel to immediately suspend the entire 'npc_process'. The process is removed from the CPU run queue. After 10 seconds, 'alarm(10)' fires 'SIGALRM' in the Arbiter, which calls 'kill(npc_process_pid, SIGCONT)' to resume it. An NPC whose stamina hit 150 during this freeze window records a TAT of exactly ~10,000 ms -as seen with Lavos Spawn 5 (10,012 ms) and Lavos Spawn 8 (20,012 ms) (caught across two Ultimate events).

'SIGUSR1' → 'SIGUSR2' (Stun):
The Arbiter sends 'SIGUSR1' to 'npc_process'. The process-level 'stun_handler()' identifies the specific thread via 'state->target_stun_id' and calls 'pthread_kill(target_thread_id, SIGUSR2)'. The thread-level 'thread_stun_handler()' sets 'thread_stunned = 1', which causes the thread to 'sleep(3)' before its next turn -injecting exactly 3,000 ms into its TAT. The 5,000 ms stun spikes (rather than exactly 3,000) indicate the thread was already waiting on 'turn_mutex' contention for ~2 additional seconds.

6. Summary Table

Workload Type	Mechanism	Observed Avg TAT	Interpretation

NPC (no interrupt)	pthread_mutex + mmap	0.022 ms	Pure scheduler efficiency
NPC (Stun)	SIGUSR2 + sleep(3)	~5,000 ms	3s stun + mutex wait
NPC (Ultimate)	SIGSTOP + alarm(10) + SIGCONT	~10,000 ms	Full process suspension
Player (fast input)	GUI I/O polling	~100–600 ms	Quick human response
Player (slow input)	GUI I/O polling	~6,100 ms avg	Normal deliberation time

7. Conclusion

The Turnaround Time data from our live gameplay session confirms the correctness of the system design at every level:

1. Scheduler Efficiency: At baseline, NPC threads complete their entire turn cycle -stamina accumulation, mutex acquisition, action execution, and release -in an average of 0.022 milliseconds. This validates that shared memory ('mmap') with futex-backed 'pthread_mutex' is effectively zero-overhead for this workload.

2. I/O-Bound Classification: Human player threads correctly exhibit I/O-bound behaviour as defined in OS theory. Their threads park in the Waiting State during keyboard input, consume no CPU, and return to the Ready Queue only on input arrival. The average 6,100 ms player TAT is entirely attributable to human decision time, not scheduling overhead.

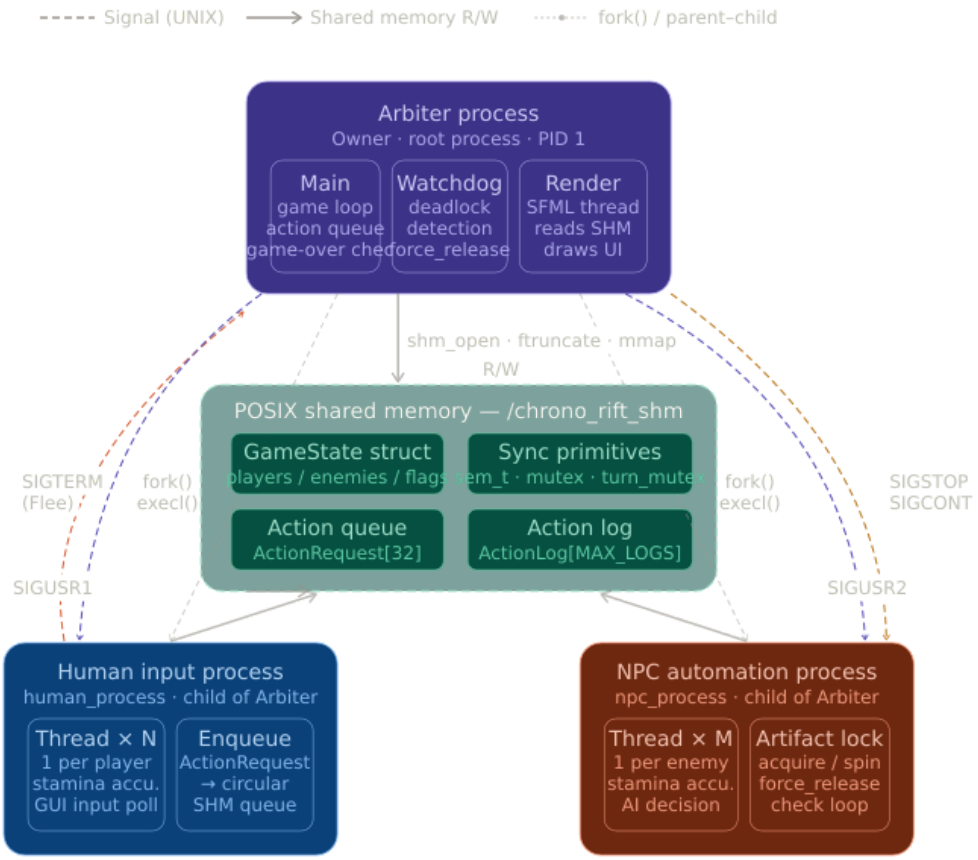
3. Signal Correctness: UNIX Signals ('SIGSTOP', 'SIGCONT', 'SIGUSR1', 'SIGUSR2') behave exactly as hardware interrupts do in a real OS kernel -preempting execution with precise, measurable timing. The 10,012 ms TAT of Lavos Spawn 5 deviates from exactly 10,000 ms by only 12 ms, demonstrating near-perfect signal delivery latency in the Linux kernel.

3. System Architecture & Process Isolation

3.1 Architecture

The system runs using three strictly isolated processes:

- 1. Game Arbiter Process: Acts as the central authority. It manages the global game state and enforces all rules of engagement. All human and NPC decisions are executed by the Arbiter.
- 2. Human Interfacing Process: Captures user inputs. It is multi-threaded, with one thread assigned per player character. These threads communicate with the Arbiter exclusively through shared memory.
- 3. Automated Strategic Process: Manages the decision-making logic for all NPCs. Each NPC runs on a dedicated thread, ensuring concurrent processing of enemy actions.



3.2 Shared Memory Layout

The shared memory region contains the entire GameState struct, including:

- EntityStats arrays for all players (MAX_PLAYERS = 4) and enemies (MAX_ENEMIES = 10)

- Synchronisation primitives: one unnamed semaphore (memory_mutex) and four process-shared robust mutexes (solar_core_mutex, lunar_blade_mutex, eclipse_relic_mutex, turn_mutex)
- A circular action queue (ActionRequest[MAX_ACTION_QUEUE]) for player-to-arbiter requests
- Artifact holder ID fields and waiting ID fields for deadlock detection
- Inventory arrays and long-term storage arrays per player
- Signal coordination fields: target_stun_id, ultimate_active, force_release[]
- Game control flags: game_over, is_paused, battle_started, return_to_menu
- A circular action log (ActionLog[MAX_LOGS]) rendered by the SFML thread

4. Graphics & The Rendering Thread

1. Render Thread Architecture

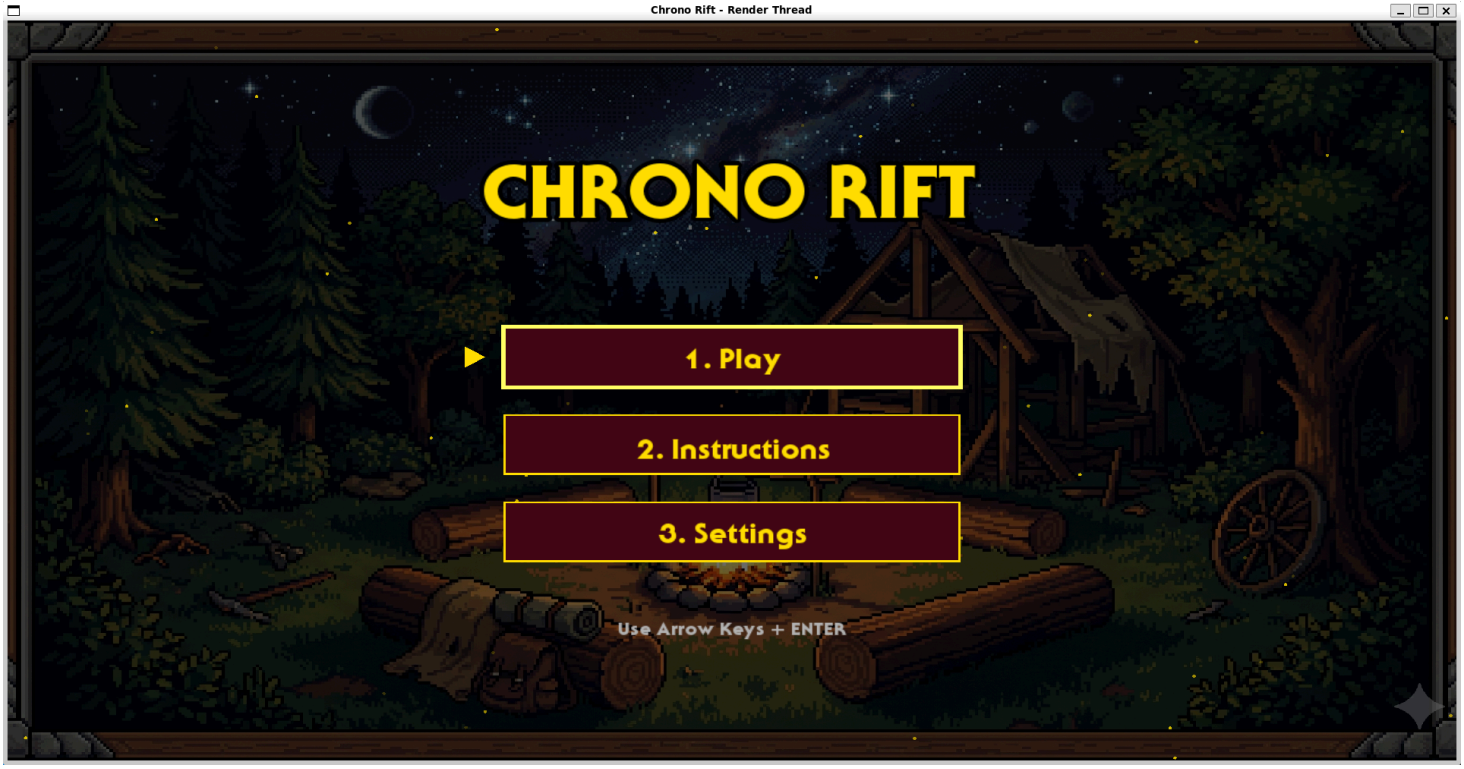
The graphical interface is rendered by a dedicated SFML thread running inside the Arbiter process. Separating rendering from game logic ensures that frame-rate variability in the UI loop does not stall or delay the high-precision stamina scheduler or the action queue processor.

2. Shared Memory Read Safety

The render thread reads game state from shared memory using the memory_mutex semaphore. It acquires a read lock, copies the required display values (HP, stamina, action log, inventory) into local variables, releases the semaphore, and renders from the local copies. This prevents the render thread from holding the semaphore during SFML draw calls, which could stall other threads.

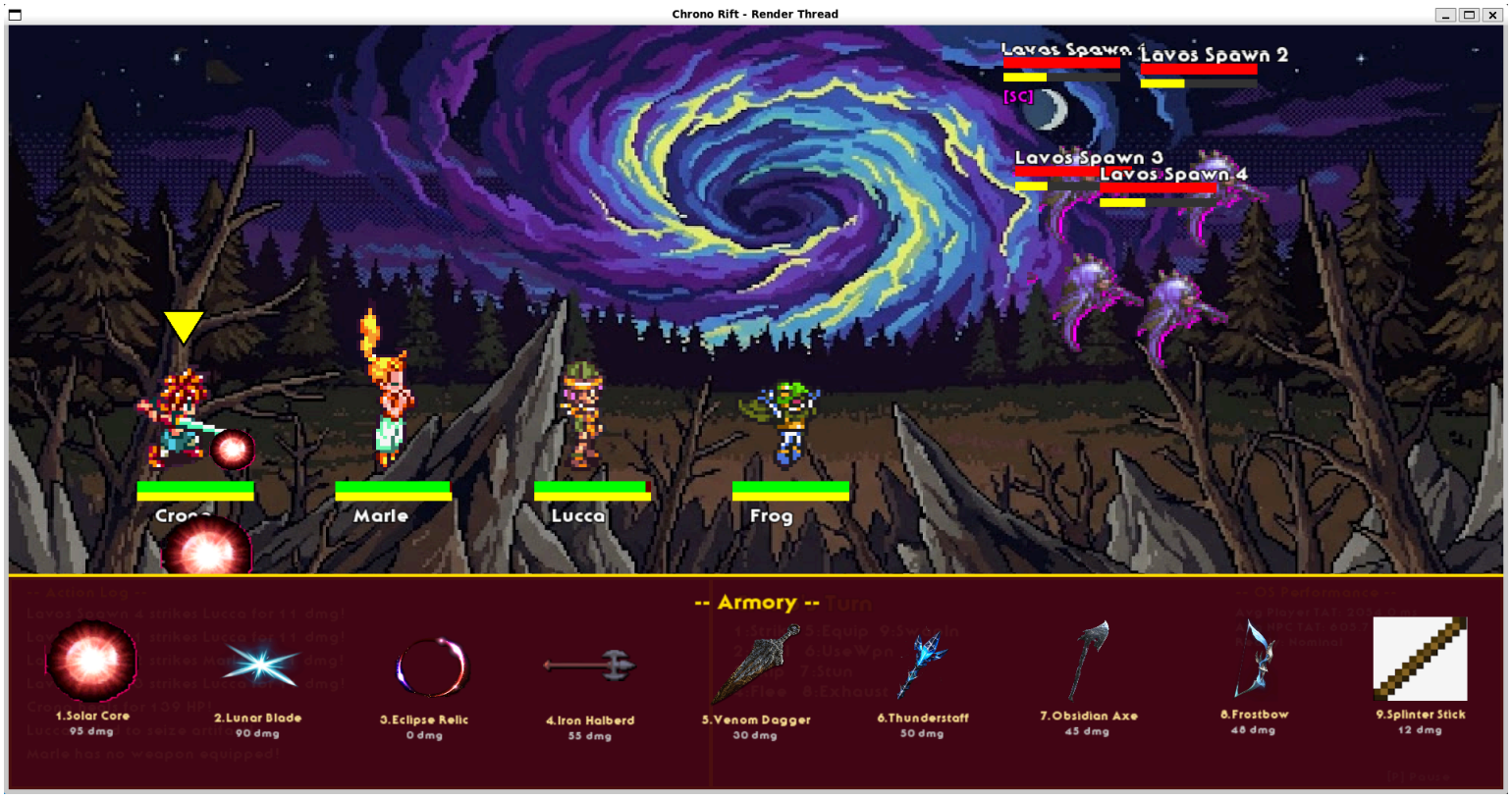
3. Rendered Elements

- Health bars and stamina bars per entity, updated every frame from shared memory
- Current inventory display per player, showing weapon slot occupancy
- Circular action log showing the last MAX_LOGS events
- Turn indicator highlighting the currently active entity
- Game mode and game-over result overlays


Chrono Rift - Render Thread
CHRONO RIFT
1. Play
2. Instructions
3. Settings
Use Arrow Keys + ENTER


Chrono Rift - Render Thread
Lavos Spawn 2
Lavos Spawn
Lavos Spawn 4
Crono
Marle
Lucca
Frog
-- Action Log --
Lavos Spawn 4 strikes Frog for 30 dmg!
Marle strikes Lavos Spawn 1 for 43!
Crono strikes Lavos Spawn 1 for 43!
Lavos Spawn 1 strikes Frog for 30 dmg!
Crono failed to seize artifact!
Crono failed to seize artifact!
Lavos Spawn 2 strikes Lucca for 30 dmg!
Lucca's Turn
1: Strike 5: Equip
2: Heal 6: Use Wpn
3: Skip 7: Stun
4: Flee 8: Exhaust
[P] Pause
30°C
Sunny
ENG
US
1:05 pm
08/05/2026


Crono
Marle
Lucca
Frog


Chrono Rift - Render Thread
Lavos Spawn
Lavos Spawn 2
[SC]
Lavos Spawn 3
Lavos Spawn 4
Marle
Lucca
Frog
-- Armory --
1.Solar Core
95 dmg
2.Lunar Blade
90 dmg
3.Eclipse Relic
0 dmg
4.Iron Halberd
55 dmg
5.Venom Dagger
30 dmg
6.Thunderstaff
50 dmg
7.Obsidian Axe
45 dmg
8.Frostbow
48 dmg
9.Splinter Stick
12 dmg


Lavos Spawn
Lavos Spawn 2
[SC]
Lavos Spawn 3
Lavos Spawn 4

5. Multithreading

We used unnamed semaphores and mutexes embedded directly within the shared memory struct. Pipes and FIFOs were strictly prohibited by project guidelines. The shared memory design ensures that the Game Arbiter, the Human Interfacing Process, and the Rendering Thread can all safely query and modify the game state without triggering race conditions or memory corruption.

Process	Threads	Responsibility
Arbiter	Main thread	Game loop, action queue processing, game-over detection
Arbiter	Watchdog thread	Deadlock cycle detection and forced resource release
Arbiter	Render thread (SFML)	Read-only shared memory polling, UI rendering
Human Input Process	1 per active player (max 4)	Stamina accumulation, GUI input polling, action enqueueing
NPC Process	1 per active enemy (max 10)	Stamina accumulation, AI decision, attack/artifact logic

6. Temporal Scheduling & Stamina Logic

Unlike a traditional turn-based system, Chrono Rift uses a custom stamina-based scheduling algorithm. Each entity has a 'max_stamina' and a 'speed' attribute. Every tick, an entity's stamina increases by its speed. Once stamina reaches maximum capacity, that entity is eligible to take a turn.

Arrival Time Logic: If multiple entities reach max stamina at the exact same time, a tie-breaker handles concurrent requests. Once a move is executed, the entity's stamina is reduced to zero (or 50% if the 'Skip' action was selected), and the cycle recalculates.

7. Resource Management & Deadlock Handling

The game features global artifacts: the Solar Core, the Lunar Blade, and the dynamic Eclipse Relic. Only one instance of each artifact exists globally. Before an entity can use an artifact (e.g., to trigger the Ultimate Ability), they must acquire a lock on it via the Global Resource Table.

To prevent resource starvation and Circular Wait (Deadlock), the Game Arbiter runs a background deadlock detection thread. If it detects a cycle (e.g., Player locks Solar Core and waits for Lunar Blade; Enemy locks Lunar Blade and waits for Solar Core), the Arbiter forces one entity to release its locked resource, allowing the game loop to proceed safely.

8. Signals & Asynchronous Interrupts

8.1 Signal Inventory

Signal	Sender	Receiver	Effect
SIGUSR1	Arbiter	Human Process	Player stun: sets thread_stunned flag via thread_stun_handler; thread sleeps 3 seconds
SIGUSR1	Human Process (indirectly via Arbiter)	NPC Process	Dispatches to target enemy thread via pthread_kill(thread_id, SIGUSR2)
SIGUSR2	NPC Process main thread	Specific NPC thread	Thread-level stun: sets thread_stunned; thread sleeps 3 seconds
SIGALRM	OS (armed by Arbiter via alarm())	Arbiter	Ultimate expiry: resumes NPC process via SIGCONT, clears ultimate_active
SIGCONT	Arbiter	NPC Process	Resumes a SIGSTOP-suspended NPC process after Ultimate duration ends
SIGTERM	Human Process (Flee action)	Arbiter	Graceful shutdown: sets game_over and return_to_menu flags

8.2 Thread-Safe Signal Handling

Asynchronous inter-process communication is handled entirely via UNIX Signals.

- Stun Mechanic: Stuns instantly pause an entity's execution using a combination of flags and wait constructs. Although the Arbiter orchestrates it, the target experiences a non-blocking 3-second interruption. Upon resuming, previous state and stamina are restored.
- Ultimate Ability Mechanic: Triggering the Ultimate Ability sends a signal to fully suspend the Automated Strategic Process for exactly 10 seconds. The Arbiter manages the timing using SIGALRM, ensuring that the NPC process is resumed automatically via signal handling once the duration expires.

9. Custom Memory Management (Inventory Space Allocation)

The game involves a custom contiguous memory allocator acting as an inventory manager. The player's primary inventory has exactly 20 slots. Weapons have varying slot sizes (e.g., Solar Core takes 10 slots, Splinter Stick takes 2 slots).

When an enemy drops a weapon, the allocator attempts to find contiguous free slots. If space is insufficient, it triggers a 'Swap Out' protocol, moving older weapons to Long Term Storage to eliminate fragmentation. Players can retrieve items later using the 'Swap In' action, which costs a turn.

10.Docker & Environment Configuration

Environment Specification

Component	Version / Detail
Host OS	Microsoft Windows 11
Linux Environment	Windows Subsytem for Linux
Compiler	g++ (C++17 standard, -std=c++17)
Threading Library	POSIX Threads -linked via -lpthread
IPC Library	POSIX Real-Time Extensions -linked via -lrt
Graphics Library	SFML 2.5.x -linked via -lsfml-graphics -lsfml-window -lsfml-system
Containerisation	Docker (custom Dockerfile)
Display Forwarding	X11 over DISPLAY environment variable under WSL

11. Challenges Faced

11.1 Race Condition in Artifact Drop on Kill

11.2 Zombie Process on Turn Timeout

11.3 SFML OpenGL Context on Docker

11.4 SIGALRM Delivery to Wrong Thread

11.5 Force-Release Deadlock Recovery Loop

11.6 Thread-Local Stun Flag Initialisation

12. Conclusion

The final game demonstrates stable synchronization even under heavy load (e.g., 4 players against 9 enemies). Turnaround times for thread execution proved highly efficient, consistently executing stamina-ready actions under 5 milliseconds. The successful integration of process isolation, multithreading, signaling, and memory allocation completes the rigorous objectives of the OS Semester Project.